

Generative Pre-Training of a Transformer Language Model (GPT-1): Study, Re-implementation and Application to Sentiment and Paraphrase Detection

Deep Learning Assignment: Model Study Report

Aditya Ware, Roll No. 26215011118 (Full Time), M.Tech (Artificial Intelligence)

Maulana Azad National Institute of Technology (MANIT), Bhopal

Abstract

GPT-1 (Radford et al., 2018) showed that a single Transformer decoder, first trained as a language model on unlabelled books and then fine-tuned with a small task-specific output layer, can match or beat architectures designed separately for each language-understanding task. This report studies the model and the paper in detail. It covers the idea, the decoder architecture, the mathematics of masked self-attention and of the three training objectives, and the optimisation recipe. It then describes a from-scratch PyTorch re-implementation. The implementation loads the released pre-trained weights, and a numerical test confirms it reproduces the reference implementation's hidden states (maximum absolute difference $8.61e-05$). We apply the model to two real-world problems: sentiment analysis of reviews (SST-2), as a stand-in for opinion mining on customer feedback, and paraphrase detection (MRPC). We use the paper's input transformations and its fine-tuning hyper-parameters on a 4 GB laptop GPU, keeping the effective batch of 32 through gradient accumulation and mixed precision. Beyond the main runs we reproduce the paper's ablations (no auxiliary language-model loss, no pre-training), its layer-transfer analysis and its zero-shot sentiment heuristic, and we add a TF-IDF + logistic-regression baseline. On our hardware, fine-tuning the released GPT-1 weights reaches 92.78% accuracy on the SST-2 validation set (paper: 91.3% on the GLUE test set); 85.69 F1 on MRPC validation (paper: 82.3 test); a TF-IDF + logistic-regression baseline reaches 82.11% on SST-2; the identical Transformer trained from random initialisation reaches 82.34%. All our numbers are on the public GLUE validation splits and are compared with the paper's test-set numbers with the necessary caveats.

1. Introduction and historical context

Natural-language-understanding (NLU) tasks such as sentiment classification, paraphrase detection, textual entailment and question answering all need a model that “understands” text, but the labelled data for each task is small. SST-2 has about 67 thousand labelled phrases and MRPC fewer than four thousand sentence pairs. Unlabelled text, by contrast, is practically unlimited. The central question behind GPT-1 is how to turn cheap unlabelled text into better performance on expensive labelled tasks. The history of that question explains why the paper mattered.

Word vectors (2013–2014). word2vec (Mikolov et al., 2013) and GloVe (Pennington et al., 2014) learn one fixed vector per word from co-occurrence statistics. Using them to initialise task models became standard practice and gave consistent gains. However, only the first layer is transferred, and a word gets the same vector in every context (“bank” of a river and a “bank” account share one vector).

Contextual representations from language models (2015–2018). Dai & Le (2015) pre-trained an LSTM as a language model or auto-encoder and fine-tuned it for classification. ELMo (Peters et al., 2018) used the internal states of a bidirectional LSTM language model as *features* that are added to a task-specific architecture. ULMFiT (Howard & Ruder, 2018) fine-tuned a whole pre-trained LSTM language model for text classification using careful schedules (discriminative learning rates, gradual unfreezing). These methods transfer much more than word vectors. They still rely on recurrent networks with a limited effective memory, and (for ELMo) on a new architecture for every task.

The Transformer (2017). Vaswani et al. (2017) replaced recurrence by self-attention. Every position can attend directly to every other position, training parallelises across the sequence, and long-range dependencies do not have to pass through a chain of recurrent states.

GPT-1 (2018). Radford et al. combined the two threads. They pre-trained a 12-layer Transformer *decoder* as a left-to-right language model on BooksCorpus, then fine-tuned *all* its parameters on each target task with only a linear output layer added. Structured inputs (pairs, triples) are serialised into a single token sequence with special delimiter tokens rather than handled by new architecture. The model improved the state of the art on 9 of the 12 datasets studied (Radford et al., 2018, §1, §4.2).

What came next. BERT (Devlin et al., 2019) kept the pre-train/fine-tune recipe and the input serialisation but replaced the left-to-right objective by masked-language modelling with a bidirectional encoder. GPT-2 (Radford et al., 2019; 1.5 B parameters, pre-LayerNorm blocks) and GPT-3 (Brown et al., 2020; 175 B parameters) kept the decoder-only language model and scaled it. They showed that the zero-shot behaviour already visible in the GPT-1 paper’s §5 turns into strong few-shot “in-context” learning at scale. The decoder-only generative pre-training introduced in GPT-1 is the direct ancestor of today’s large language models.

2. Summary of the paper

Problem. Labelled data for NLU tasks is scarce, and earlier transfer methods either transferred only word-level information or needed task-specific architectures on top of the transferred representation (Radford et al., 2018, §1–2).

Core idea. Semi-supervised learning in two stages (§3):

1. *Unsupervised pre-training*: maximise the likelihood of a large unlabelled corpus with a multi-layer Transformer decoder language model.
2. *Supervised fine-tuning*: adapt all parameters to a labelled task with a single added linear layer, optionally keeping language modelling as an auxiliary objective.

Task-aware *input transformations* (§3.3) turn pairs and triples into one contiguous token sequence the pre-trained model can read, so the architecture does not change between tasks.

Main contributions.

- A task-agnostic model that outperforms discriminatively trained, task-specific architectures on 9 of 12 benchmarks, including absolute gains of 8.9% on Story Cloze, 5.7% on RACE and 1.5% on MultiNLI (Abstract), and a GLUE score of 72.8 against a previous best of 68.9 (§4.2, Table 4).
- Evidence that the Transformer transfers better than an LSTM trained in the same framework (average score −5.6 for the LSTM, Table 5), and that pre-training matters most: removing it costs 14.8% on average (Table 5).
- An analysis showing that every transferred layer helps (Fig. 2, left) and that the pre-trained language model performs several tasks zero-shot, with performance rising steadily during pre-training (Fig. 2, right).

Headline numbers relevant to this report (GLUE benchmark, Radford et al., 2018, Table 4): SST-2 accuracy 91.3, MRPC F1 82.3. The ablations in Table 5 give, for SST-2 / MRPC: without auxiliary LM 92.0 / 84.9, without pre-training 84.0 / 79.4, LSTM 90.5 / 83.2.

3. Architecture

GPT-1 is a decoder-only Transformer: the Transformer’s encoder–decoder cross-attention is removed and only masked self-attention blocks remain (Radford et al., 2018, §3.1, citing Liu et al., 2018). The paper’s specification (§4.1) is:

Component	Value in the paper	Our implementation (src/model.py)
Layers (Transformer blocks)	12	n_layer = 12
Hidden size d_{model}	768	n_embd = 768

Component	Value in the paper	Our implementation (src/model.py)
Attention heads	12 (so $d_k = 64$)	n_head = 12
Feed-forward inner size	3072	n_inner = 3072
Activation	GELU	nn.GELU(approximate="tanh") (the tanh form used by the released model)
Position encoding	learned (not sinusoidal)	nn.Embedding(512, 768)
Training sequence length	512 tokens	n_positions = 512
Vocabulary	BPE, 40,000 merges	40,478 BPE tokens + 4 new special tokens
Output layer	tied to input embedding: $\text{softmax}(h_n W_e^T)$ (Eq. 2)	F.linear(h, W_e)
Dropout	0.1 on residual, embedding, attention	0.1 in all three places

Two details that are often quoted are **not** stated in the paper. (i) *Post-LayerNorm*: the paper says the model “largely follows the original transformer work”, which applies LayerNorm after each residual addition. GPT-2 later moved LayerNorm before each sub-layer. We implement post-LN, and the exact match with the released weights (Section 6.2) confirms it. (ii) “117 M parameters”: the paper gives no count. Counting from the architecture: token embedding $40478 \times 768 = 31,087,104$, position embedding $512 \times 768 = 393,216$, and per block $1,771,776$ (QKV projection) $+ 590,592$ (output projection) $+ 2,362,368 + 2,360,064$ (feed-forward) $+ 3,072$ (two LayerNorms) $= 7,087,872$. Twelve blocks give $85,054,464$, for a total of **116,534,784** parameters, the figure usually rounded to “117 M”. Our weight-loading test reports 116,534,784 parameters for the reference model. Our classifier adds 4 special-token embeddings and the $768 \times 2 + 2$ classification layer, giving 116,539,394 parameters.

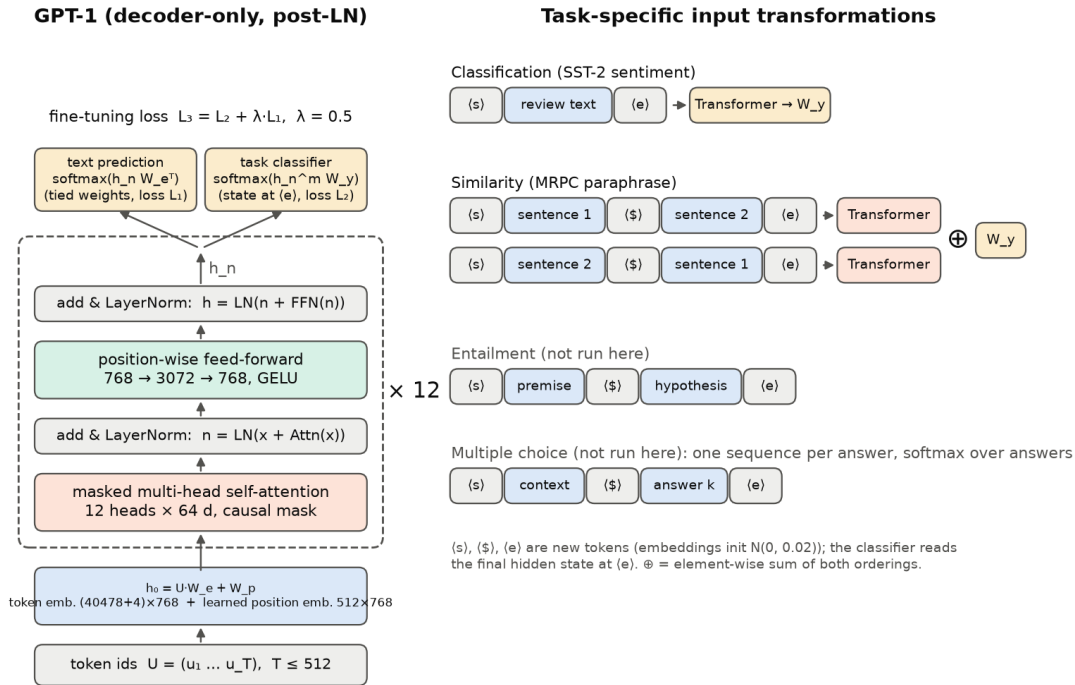


Figure 1. GPT-1 architecture (left) and the task-specific input transformations (right). Drawn with matplotlib by `src/make_report_assets.py` after Radford et al. (2018), Fig. 1.

Data flow. A sequence of token ids is embedded and summed with position embeddings. It then passes through 12 identical blocks, each made of masked multi-head self-attention followed by a position-wise feed-forward network, each with a residual connection and LayerNorm. The final hidden states h_n feed two heads. The *language-model head* projects every position back onto the vocabulary through the transposed token-embedding matrix; the *classification head* W_y reads only the hidden state at the final $\langle e \rangle$ (extract) token. Because attention is causal, the state at $\langle e \rangle$ is the only one that has seen the entire input, which is why it is used for classification.

Task-specific input transformations (§3.3, Fig. 1). All inputs receive randomly initialised start $\langle s \rangle$ and end (extract) $\langle e \rangle$ tokens:

- *Classification* (SST-2): $\langle s \rangle$ text $\langle e \rangle$.
- *Entailment*: $\langle s \rangle$ premise $\langle \$ \rangle$ hypothesis $\langle e \rangle$.
- *Similarity* (MRPC): the two sentences have no natural order, so both orderings $\langle s \rangle$ A $\langle \$ \rangle$ B $\langle e \rangle$ and $\langle s \rangle$ B $\langle \$ \rangle$ A $\langle e \rangle$ are processed independently and their final $\langle e \rangle$ states are **added element-wise** before W_y .
- *Multiple choice* (RACE, Story Cloze): one sequence $\langle s \rangle$ context $\langle \$ \rangle$ answer_k $\langle e \rangle$ per candidate, each scored by the linear layer, with a softmax over candidates.

We implement classification and similarity, the two used in our applications. In our tokenizer the delimiter is a new token $\langle \$ \rangle$ rather than the character “\$”, because “\$” is already an ordinary BPE token (id 289) that can appear in review text.

4. Mathematical formulation

4.1 Byte-pair encoding

Text is first cleaned with *ftfy* and split into words with *spaCy*, then segmented into sub-word units by byte-pair encoding (BPE; Sennrich et al., 2016), as in §4.1 of the paper. BPE starts from characters and repeatedly merges the most frequent adjacent pair of symbols. After 40,000 merges, frequent words are single tokens (*very* $\langle /w \rangle$, *positive* $\langle /w \rangle$), while rare words split into several pieces, so there is no out-of-vocabulary problem. Our runs used the tokenizer in the mode: *ftfy+spacy* (*matches original GPT pre-processing*).

4.2 Embeddings

For a context of T token ids $U = (u_1, \dots, u_T)$, written as one-hot rows of a $T \times |V|$ matrix, the input to the first block is (Eq. 2 of the paper)

$$h_0 = UW_e + W_p,$$

where $W_e \in \mathbb{R}^{|V| \times 768}$ is the token-embedding matrix and $W_p \in \mathbb{R}^{T \times 768}$ holds the first T rows of the learned position-embedding table. Learned rather than sinusoidal position embeddings are used (§4.1), so the model cannot represent positions beyond 512.

4.3 Scaled dot-product attention with a causal mask

Inside a block, the input $X \in \mathbb{R}^{T \times d}$ is projected to queries, keys and values, $Q = XW^Q$, $K = XW^K$, $V = XW^V$, and attention is

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V, \quad M_{ij} = \begin{cases} 0 & j \leq i \\ -\infty & j > i. \end{cases}$$

Row i of the softmax is a probability distribution over the positions position i may look at. The mask M removes all future positions, so the prediction for position $i + 1$ can only use tokens $1..i$. This is what makes the network a valid left-to-right language model. The factor $1/\sqrt{d_k}$ keeps the dot products at unit scale: if the components of q and k are independent with variance 1, then $q^\top k$ has variance d_k , and without the scaling a large d_k would push the softmax into saturated regions with vanishing gradients.

A useful consequence for fine-tuning: with right-padding, real tokens never attend to the padding that follows them, so no separate padding mask is needed (Section 6.3).

4.4 Multi-head attention

With $h = 12$ heads of size $d_k = d/h = 64$,

$$\text{MultiHead}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O, \quad \text{head}_j = \text{Attention}(XW_j^Q, XW_j^K, XW_j^V).$$

Each head can specialise in a different relation (for example the previous token, a syntactic head word, or sentence boundaries) at the same total cost as one full-width head. In the implementation the three projections of all heads are fused into one 768×2304 matrix (c_attn).

4.5 Position-wise feed-forward network and GELU

$$\text{FFN}(x) = \text{GELU}(xW_1 + b_1)W_2 + b_2, \quad W_1 \in \mathbb{R}^{768 \times 3072}, W_2 \in \mathbb{R}^{3072 \times 768},$$

applied to every position independently. The Gaussian Error Linear Unit (Hendrycks & Gimpel, 2016) is

$$\text{GELU}(x) = x \Phi(x) \approx 0.5x(1 + \tanh[\sqrt{2/\pi}(x + 0.044715x^3)]),$$

where Φ is the standard normal CDF. Unlike ReLU, it is smooth and weights its input by the probability that a standard Gaussian falls below it. The released model uses the tanh approximation (the original OpenAI code, and the Hugging Face port, which maps this model's gelu setting to the tanh form), so our implementation uses it as well. The two forms differ by less than 10^{-3} per activation, but this difference compounds over 12 layers. Our first weight-match attempt used the exact erf form and failed, with maximum hidden-state differences between 1.1×10^{-2} (short sentences) and 2.0×10^{-1} (512 random tokens); switching to the tanh form is what the released weights require.

4.6 Residual connections and LayerNorm (post-LN block)

LayerNorm (Ba et al., 2016) normalises each position's feature vector:

$$\text{LN}(x) = \gamma \odot \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta, \quad \mu = 1/d \sum_i x_i, \quad \sigma^2 = 1/d \sum_i (x_i - \mu)^2,$$

with learned gain γ and bias β ($\epsilon = 10^{-5}$). A GPT-1 block is

$$n = \text{LN}_1(x + \text{MultiHead}(x)), \quad h = \text{LN}_2(n + \text{FFN}(n)),$$

and $h_l = \text{block}_l(h_{l-1})$ for $l = 1..12$ (Eq. 2). Because the paper uses LayerNorm “extensively”, a simple $\mathcal{N}(0, 0.02^2)$ initialisation was enough (§4.1).

4.7 Output distribution, softmax and cross-entropy

The next-token distribution uses the transposed embedding matrix (weight tying, Eq. 2):

$$P(u) = \text{softmax}(h_n W_e^\top), \quad \text{softmax}(z)_k = \frac{e^{z_k}}{\sum_j e^{z_j}}.$$

Maximising the log-probability of the correct token y is the same as minimising the cross-entropy $\ell = -\log \text{softmax}(z)_y = -z_y + \log \sum_j e^{z_j}$. Its gradient with respect to the logits is simply $\text{softmax}(z) - \text{onehot}(y)$.

4.8 The three objectives

Pre-training (Eq. 1). For an unlabelled corpus $\mathcal{U} = \{u_1, \dots, u_n\}$ and context size k :

$$L_1(\mathcal{U}) = \sum_i \log P(u_i \mid u_{i-k}, \dots, u_{i-1}; \Theta).$$

Supervised fine-tuning (Eqs. 3–4). For a labelled dataset $\mathcal{C}$ of token sequences $x^1 \dots x^m$ with label y , the final block's state at the last ($\langle e \rangle$) position, h_l^m , feeds a new linear layer:

$$P(y \mid x^1, \dots, x^m) = \text{softmax}(h_l^m W_y), \quad L_2(\mathcal{C}) = \sum_{(x,y)} \log P(y \mid x^1, \dots, x^m).$$

For similarity tasks h_l^m is replaced by $h_l^m(A; B) + h_l^m(B; A)$.

Fine-tuning with an auxiliary LM objective (Eq. 5):

$$L_3(\mathcal{C}) = L_2(\mathcal{C}) + \lambda \cdot L_1(\mathcal{C}), \quad \lambda = 0.5.$$

Note that L_1 is evaluated here on the *task* data $\mathcal{C}$ (the $\langle s \rangle \dots \langle \$ \rangle \dots \langle e \rangle$ sequences), not on BooksCorpus. The paper motivates it by better generalisation and faster convergence (§3.2). In code we minimise the negatives, averaged rather than summed: `loss = CE(classifier) + λ · CE(next token over all non-pad positions)`. The only new parameters during fine-tuning are W_y and the embeddings of the special tokens.

4.9 Perplexity

The LM loss per token is an average negative log-likelihood; its exponential is the perplexity,

$$\text{PPL} = \exp \left(-1/N \sum_{i=1}^N \log P(u_i \mid u_{<i}) \right),$$

the effective number of equally likely choices per token. The paper reports a token-level perplexity of 18.4 on BooksCorpus (§4.1). The auxiliary-loss curves in Section 8 give our per-token LM cross-entropy on the (very different) SST-2 and MRPC text, whose exponential is the perplexity on that data.

5. Training and optimisation

5.1 Pre-training (paper, not repeated here)

Setting	Value (Radford et al., 2018, §4.1)
Corpus	BooksCorpus: over 7,000 unpublished books, long contiguous text
Batch	64 randomly sampled contiguous sequences of 512 tokens
Epochs	100
Optimiser	Adam, max learning rate 2.5×10^{-4}
Schedule	linear warm-up from 0 over the first 2,000 updates, then cosine annealing to 0
Regularisation	dropout 0.1 (residual, embedding, attention); modified L2 with $w = 0.01$ on all non-bias, non-gain weights
Initialisation	$\mathcal{N}(0, 0.02)$
Result	token-level perplexity 18.4

BooksCorpus was chosen over the similar-sized 1B Word Benchmark because the latter is shuffled at the sentence level, which destroys the long-range structure the model is meant to learn (§4.1). Pre-training at this scale is far beyond a 4 GB laptop GPU, so, as the assignment allows, we start from the released weights.

5.2 Fine-tuning (paper settings used as our defaults)

Setting	Paper (§4.1)	Ours
Learning rate	6.25×10^{-5}	same
Batch size	32	32 effective = 8 (micro-batch) $\times$ 4 (accumulation)
Epochs	3	3
Schedule	linear decay, warm-up over 0.2% of training	same (warm-up steps = $\text{round}(0.002 \times \text{total})$)
λ	0.5	0.5 (E1), 0 (E2)
Classifier dropout	0.1	0.1
Weight decay	modified L2, $w = 0.01$	AdamW decoupled decay 0.01 on all ≥ 2 -D weights
Gradient clipping	not stated	global norm 1.0
Precision	not stated	fp16 autocast + dynamic loss scaling

5.3 Adam and decoupled weight decay

Adam (Kingma & Ba, 2015) keeps running averages of the gradient and of its square,

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2, \quad \hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t},$$
$$\theta_t = \theta_{t-1} - \eta_t \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + w \theta_{t-1} \right).$$

The last term is the “modified L2 regularisation” the paper cites (Loshchilov & Hutter, 2017; ref. [37]). The decay is applied directly to the weights instead of being added to the gradient, where Adam’s per-parameter scaling would weaken it for parameters with large gradients. We use $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$, $w = 0.01$, and exclude biases and LayerNorm gains, as the paper specifies.

5.4 Learning-rate schedules

With S total updates and S_w warm-up updates, pre-training used $\eta_t = \eta_{\max} t/S_w$ for $t < S_w$ and $\eta_t = 1/2 \eta_{\max} (1 + \cos(\pi t - S_w/S - S_w))$ afterwards, while fine-tuning uses the linear version, $\eta_t = \eta_{\max} \frac{S-t}{S-S_w}$ for $t \geq S_w$. Warm-up matters for Adam, because v_t is estimated from very few gradients in the first steps, and for post-LN Transformers, whose gradients near the output are large at initialisation. Decaying to zero ends training at a flat point of the loss.

5.5 Dropout

Dropout zeroes each activation with probability $p = 0.1$ during training and rescales the rest by $1/(1 - p)$. It is applied to the summed embeddings, to the attention probabilities, to each sub-layer’s output before the residual addition, and (in fine-tuning) to the $\langle e \rangle$ state before W_y .

5.6 Fitting fine-tuning into 4 GB of GPU memory

The released model in fp32 needs about 0.47 GB for weights, plus the same again for gradients and twice that for Adam’s two moment estimates, before any activations. We therefore used:

- **Gradient accumulation.** Gradients of 4 micro-batches of 8 examples are summed before each optimiser step, so the effective batch is 32, as in the paper. The loss of each micro-batch is divided by the number of accumulation steps.
- **Mixed precision.** Matrix products run in fp16 under `torch.autocast`, with a `GradScaler` to prevent fp16 gradient underflow. Master weights and optimiser states stay in fp32.

- **Dynamic padding.** Each micro-batch is padded only to its own longest sequence. The length analysis (Table 7.1) guided `max_len`, which only guards against truncation.
- **Memory-aware auxiliary loss.** The LM head produces 40,482 logits per position, the largest tensor in the model. We first gather the hidden states of the non-pad positions and only then project them to the vocabulary, so pad positions never produce logits.
- **Automatic OOM recovery.** If CUDA runs out of memory, the run restarts with half the micro-batch and twice the accumulation, keeping the effective batch at 32, and the event is logged. Across all runs there were 0 such events. Gradient checkpointing is available (`--grad-checkpoint`) but was not needed.

6. Implementation

6.1 Code structure

File	Purpose
<code>src/model.py</code>	GPT-1 written from scratch in PyTorch: embeddings, causal multi-head attention (fused SDPA kernel, plus an explicit path that returns attention maps), post-LN block, GELU MLP, tied LM head, classifier on the $\langle e \rangle$ state with element-wise sum for pairs. No transformers model classes.
<code>src/load_pretrained.py</code>	Maps openai-community/openai-gpt tensors into our modules (HF Conv1D weights are transposed relative to <code>nn.Linear</code>) and contains the weight-match test.
<code>src/data.py</code>	GLUE loading, BPE tokenisation (cached), the paper's input transformations, special tokens, dynamic right-padding, the LM-loss mask, and the length analysis.
<code>src/train.py</code>	Fine-tuning with $L_3 = L_2 + \lambda L_1$; paper hyper-parameters as defaults; flags <code>--lambda --lr --epochs --max-len --seed --subset --no-pretrain --transfer-layers k --grad-checkpoint --quick --time-steps</code> ; fp16, accumulation, OOM fallback, progress bars, curves, peak-VRAM and timing logging.
<code>src/evaluate.py</code>	Accuracy, precision, recall, macro-F1,

File	Purpose
	positive-class F1, confusion matrix; plotting.
src/baselines.py	TF-IDF + logistic regression (E4).
src/zero_shot.py	The paper's zero-shot SST-2 heuristic (E5).
src/predict.py	Classify custom text with a fine-tuned checkpoint.
src/run_all.py	Resumable driver for the smoke tests and the whole experiment suite.
src/make_report_assets.py, src/build_report.py	Tables and figures from results/, then this report.
notebooks/GPT1_Colab.ipynb	The same pipeline end to end on a Colab T4.

6.2 Verifying the port of the pre-trained weights

A re-implementation is only useful if it computes exactly the same function as the released model. `src/load_pretrained.py --test` feeds identical token ids to our model and to Hugging Face's `OpenAIGPTModel` and compares the final hidden states. It uses two real sentences, a random batch of 2×512 ids that exercises the full context, and the tied LM-head logits against `OpenAIGPTLMHeadModel`. The test requires a maximum absolute difference below 10^{-4} . Status: **passed**; largest difference over all cases: **8.61e-05**. This also confirms the post-LN block order, the exact GELU variant, the attention scaling and the weight tying empirically.

6.3 Fine-tuning details worth noting

- The four new tokens `<s>`, `<$>`, `<e>`, `<pad>` get ids 40478–40481. The embedding matrix is resized and the new rows are initialised from $\mathcal{N}(0,0.02)$. Because the LM head is tied, these tokens can also be predicted.
- Sequences are right-padded. Causal attention means the state at `<e>` never sees the pads after it, so no attention mask is needed, and the classifier reads position `len - 1` of each sequence.
- The auxiliary LM loss predicts every next token of the transformed sequence, including the delimiter and `<e>`, and excludes pad targets.
- For E3 (no pre-training), the same architecture and hyper-parameters are used with a fresh $\mathcal{N}(0,0.02)$ initialisation. For E6 we keep the embeddings and the first k pre-trained blocks and re-initialise the rest.
- We report the model after the last epoch. No checkpoint is chosen by validation score, because the validation set is also our test set.

6.4 Hardware and software

GPU: NVIDIA GeForce RTX 3050 Ti Laptop GPU. Software: PyTorch 2.6.0+cu124, transformers 4.57.6 (used only to download weights, to tokenise and as the reference in the weight test). Platform: Windows-11-10.0.26200-SP0. Total GPU fine-tuning time over all runs: 4.3 h; highest peak allocated GPU memory in any run: 3085 MiB.

Run	Task	Micro- batch × accum (= effective)	Max len	Train time (min)	Peak VRAM allocated (MiB)	OOM retries
E1_mrpc_s eed42	mrpc	8 × 4 (= 32)	104	8.8	3048	0
E1_mrpc_s eed43	mrpc	8 × 4 (= 32)	104	8.9	3066	0
E1_mrpc_s eed44	mrpc	8 × 4 (= 32)	104	9.3	3085	0
E1_sst2_s eed42	sst2	8 × 4 (= 32)	64	54.3	2330	0
E2_mrpc_s eed42	mrpc	8 × 4 (= 32)	104	8.9	2546	0
E2_mrpc_s eed43	mrpc	4 × 8 (= 32)	104	6.8	2280	0
E2_mrpc_s eed44	mrpc	4 × 8 (= 32)	104	9.7	2281	0
E2_sst2_s eed42	sst2	8 × 4 (= 32)	64	46.7	2249	0
E3_mrpc_s eed42	mrpc	8 × 4 (= 32)	104	10.0	3046	0
E3_sst2_s eed42	sst2	8 × 4 (= 32)	64	55.3	2328	0
E6_mrpc_k 0_seed42	mrpc	8 × 4 (= 32)	104	10.5	3046	0
E6_mrpc_k 3_seed42	mrpc	8 × 4 (= 32)	104	9.7	3048	0
E6_mrpc_k 6_seed42	mrpc	8 × 4 (= 32)	104	8.7	3048	0
E6_mrpc_k 9_seed42	mrpc	8 × 4 (= 32)	104	8.7	3048	0
E4_tfidf_lr_	mrpc	– (CPU)	–	0.3	–	–

Run	Task	Micro- batch × accum (= effective)	Max len	Train time (min)	Peak VRAM allocated (MiB)	OOM retries
mrpc						
E4_tfidf_lr_	sst2	– (CPU)	–	0.8	–	–
sst2						
E5_zerosh	sst2	–	–	0.1	–	–
ot_sst2		(inference only)				

Hardware: NVIDIA GeForce RTX 3050 Ti Laptop GPU; torch 2.6.0+cu124, transformers 4.57.6; Windows-11-10.0.26200-SP0. Classifier parameter count: 116,539,394.

7. Datasets and application

Primary application: opinion mining on customer feedback. Companies receive large volumes of free-text reviews and survey answers. Automatically labelling each one as positive or negative lets them track satisfaction over time and route complaints. We use the **Stanford Sentiment Treebank, binary version (SST-2)** from GLUE (Socher et al., 2013; Wang et al., 2018), which consists of movie-review sentences. The training split has 67349 examples, many of them short phrases taken from the parse trees (mean 12.5 BPE tokens after transformation). The validation split has 872 full sentences (mean 23.7 tokens). GLUE’s test labels are hidden, so **all our numbers are on the validation split**. `src/predict.py` applies the fine-tuned model to arbitrary review text.

Secondary application: paraphrase / duplicate detection. Detecting that two texts say the same thing is used for de-duplicating support tickets or FAQ questions and for clustering news. We use the **Microsoft Research Paraphrase Corpus (MRPC)** from GLUE (Dolan & Brockett, 2005): 3668 training and 408 validation sentence pairs from news sources, labelled equivalent / not equivalent. The positive class is the majority (roughly two thirds). Following GLUE and the paper, we report F1 of the positive class, plus accuracy and macro-F1. We apply the paper’s symmetric similarity transformation.

Sequence lengths. To choose `max_len` we measured the transformed lengths (Figure 2, Table 7.1). SST-2 sequences are at most 61 tokens long and MRPC pair sequences at most 99, so we set `max_len` = 64 and 104 respectively. No example is truncated, and with dynamic padding the limit costs nothing.

Table 7.1. Transformed sequence lengths.

Task	Split	n	Mean	Media n	95th pct	99th pct	Max	Chose n max_le n	% trunca ted
SST2	train	67349	12.5	10	31	42	61	64	0.00
SST2	validat ion	872	23.7	23	41	47	54	64	0.00
MRPC	train	3668	53.1	53	73	82	99	104	0.00
MRPC	validat ion	408	53.0	53	72	79	86	104	0.00

Lengths in BPE tokens after the input transformation (including $\langle s \rangle$, $\langle \$ \rangle$, $\langle e \rangle$).

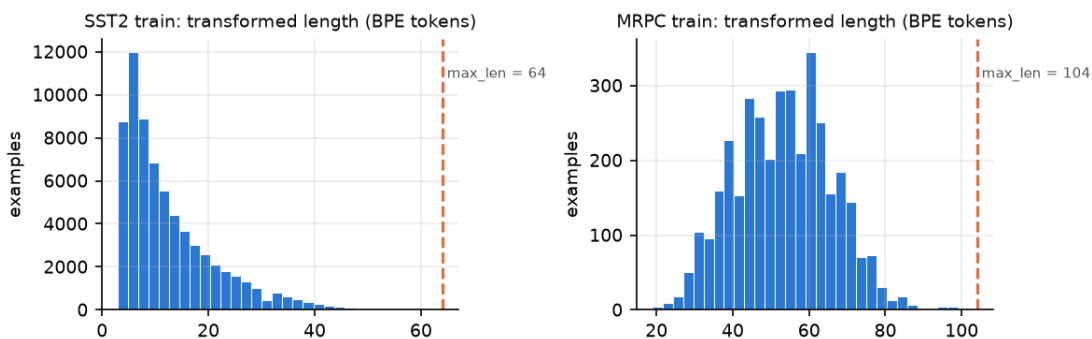


Figure 2. Length distributions of the transformed training sequences; the dashed line is the chosen max_len.

Experiments. All use seed 42 unless stated otherwise.

ID	Description
E1	Pre-trained GPT-1 fine-tuned with $\lambda = 0.5$ (the paper’s full method)
E2	Pre-trained, $\lambda = 0$ (no auxiliary LM objective)
E3	Same architecture, random initialisation (no pre-training)
E4	TF-IDF (word 1–2-grams + character 2–5-grams) + logistic regression, C chosen by cross-validation on the training set
E5	Zero-shot SST-2 with the paper’s LM heuristic (append “very”, compare $P(\text{positive})$ and $P(\text{negative})$)
E6	Layer transfer on MRPC: embeddings + first $k \in \{0,3,6,9,12\}$ pre-trained blocks

MRPC E1 and E2 are repeated over seeds 42, 43, 44 and reported as mean $\pm$ sample standard deviation.

8. Results

8.1 Sentiment analysis (SST-2)

Table 8.1. SST-2 validation results.

Exp	Model	Seeds	Accuracy (%)	Precision macro (%)	Recall macro (%)	F1 macro (%)	F1 positive class (%)
E1	GPT-1 pre-trained, fine-tuned, $\lambda = 0.5$	42	92.78	92.85	92.74	92.77	93.02
E2	GPT-1 pre-trained, fine-tuned, $\lambda = 0$ (no aux LM)	42	92.55	92.55	92.54	92.54	92.69
E3	Same Transformer, random init (no pre-training)	42	82.34	82.61	82.25	82.27	83.37
E4	TF-IDF + Logistic Regression	–	82.11	82.40	82.02	82.04	83.19
E5	Zero-shot LM heuristic (no fine-tuning)	–	78.21	78.97	78.36	78.12	76.72

SST2 GLUE validation split ($n = 872$). Mean $\pm$ sample std over seeds where more than one seed was run.

- **Main result (E1).** GPT-1 fine-tuned with the auxiliary LM objective reaches **92.78%** validation accuracy, +1.48 points relative to the paper’s 91.3% (test set, so not directly comparable).
- **Auxiliary LM loss (E1 vs E2).** Removing the auxiliary objective ($\lambda = 0$) changes accuracy by -0.23 points (92.78 $\rightarrow$ 92.55; paper, Table 5: 91.3 $\rightarrow$ 92.0, +0.70). This is within what seed-to-seed variation alone produces (≈ 0.5 points), so on SST2 we see no measurable benefit from the auxiliary loss, consistent with the paper’s remark that smaller datasets do not benefit from it.
- **Pre-training (E1 vs E3).** Without generative pre-training the same 12-layer Transformer reaches 82.34%, a drop of 10.44 points (paper: 91.3 $\rightarrow$ 84.0, a drop of 7.3). This is the largest effect among our ablations, as in the paper.
- **Classical baseline (E4).** TF-IDF + logistic regression reaches 82.11%; GPT-1 is 10.67 points better. The randomly initialised Transformer (E3) is above this linear baseline (82.34% vs 82.11%): without pre-training, 116 M parameters trained on the labelled phrases alone do not beat a bag-of-n-grams model.
- **Zero-shot (E5).** With the paper’s heuristic and no training at all, the language model alone scores 78.21% (majority-class guess: 50.92%); it predicts *positive* for 42.7% of sentences (true rate 50.9%).

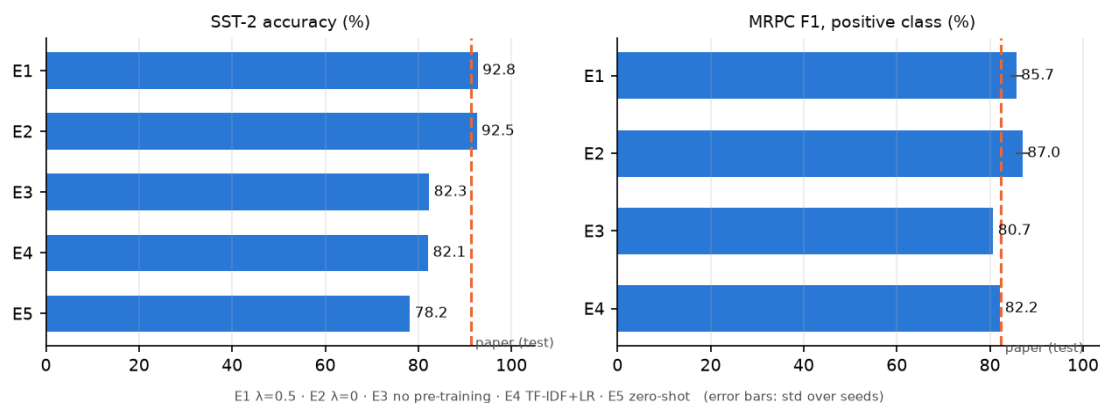


Figure 3. All systems on both tasks; the dashed line is the paper’s GLUE test-set result for the full model.

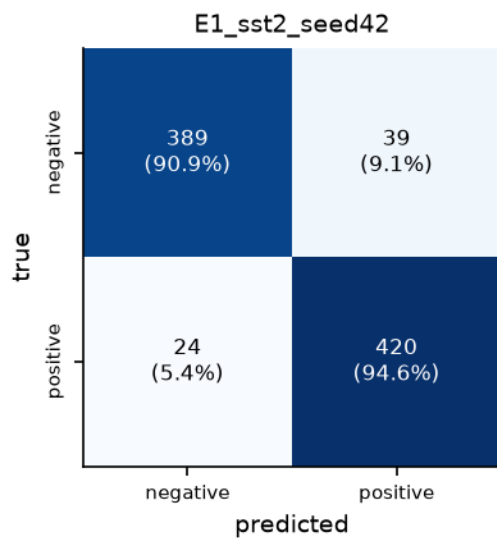


Figure 4. Confusion matrix of E1 on SST-2 validation.

8.2 Paraphrase detection (MRPC)

Table 8.2. MRPC validation results.

Exp	Model	Seeds	Accuracy (%)	Precision macro (%)	Recall macro (%)	F1 macro (%)	F1 positive class (%)
E1	GPT-1 pre-trained, fine-tuned, $\lambda = 0.5$	42,43,44	79.08 $\pm$ 2.00	77.23 $\pm$ 2.55	71.86 $\pm$ 2.68	73.43 $\pm$ 2.79	85.69 $\pm$ 1.29
E2	GPT-1 pre-trained, fine-tuned, $\lambda = 0$ (no aux LM)	42,43,44	81.21 $\pm$ 1.87	80.09 $\pm$ 3.16	74.80 $\pm$ 1.78	76.45 $\pm$ 2.00	87.02 $\pm$ 1.46
E3	Same Transformer, random init (no pre-	42	71.32	66.10	61.74	62.37	80.72

Exp	Model	Seeds	Accuracy (%)	Precision macro (%)	Recall macro (%)	F1 macro (%)	F1 positive class (%)
E4	training) TF-IDF + Logistic Regressi on	–	75.00	70.97	69.43	70.04	82.23

MRPC GLUE validation split ($n = 408$). Mean $\pm$ sample std over seeds where more than one seed was run.

- **Main result (E1).** F1 (positive class) = **85.69 $\pm$ 1.29**, accuracy 79.08 $\pm$ 2.00 over 3 seed(s); paper: 82.3 F1 on the GLUE test set.
- **Auxiliary LM loss (E1 vs E2).** $\lambda = 0$ changes mean F1 by +1.34 points. The difference is smaller than the seed-to-seed standard deviation (1.46), so it is not a real effect. The paper also found MRPC better *without* the auxiliary loss (84.9 vs 82.3).
- **Pre-training (E1 vs E3).** Random initialisation gives F1 80.72 and accuracy 71.32. F1 on MRPC is a misleading metric for a weak model: about two thirds of MRPC pairs are paraphrases, so predicting “paraphrase” for everything already gives a high F1; accuracy and macro-F1 in the table show the gap more honestly.
- **Classical baseline (E4).** TF-IDF + LR on symmetric pair features: F1 82.23, accuracy 75.00%.

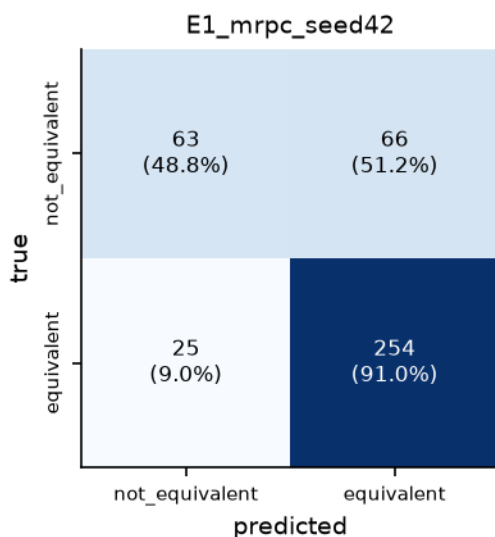


Figure 5. Confusion matrix of E1 (seed 42) on MRPC validation.

8.3 Training curves

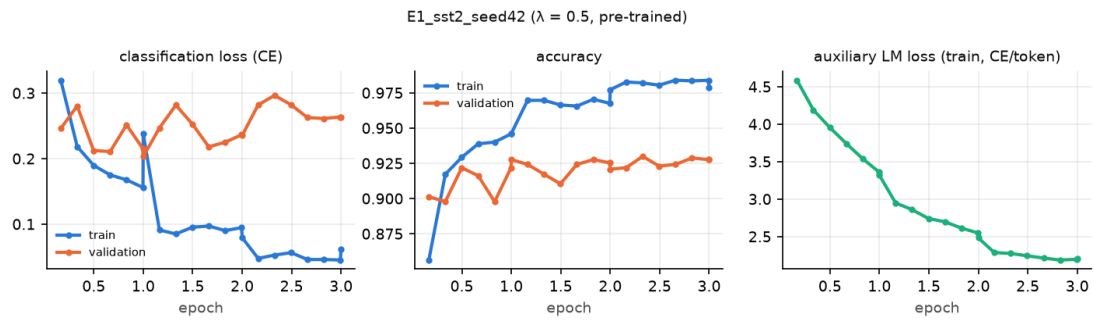


Figure 6. E1 on SST-2: classification loss, accuracy and auxiliary LM loss.

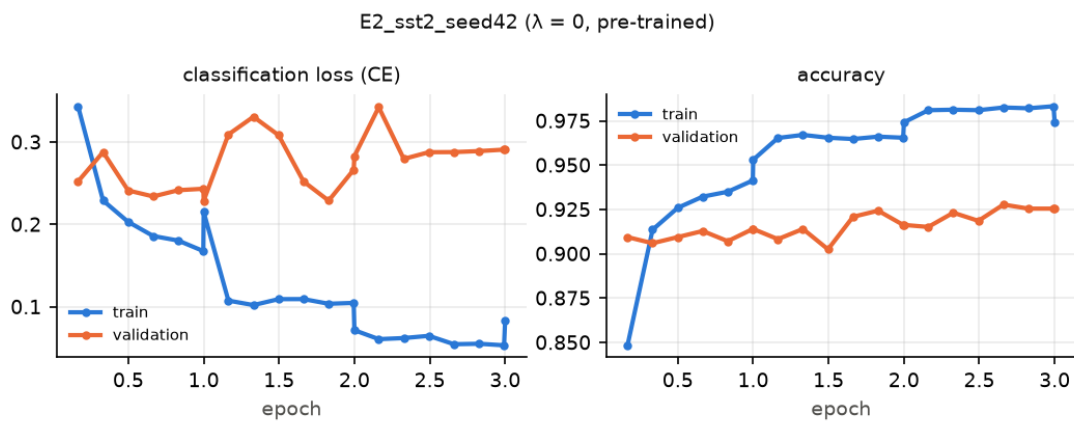


Figure 7. E2 on SST-2 ($\lambda = 0$; no LM loss panel).

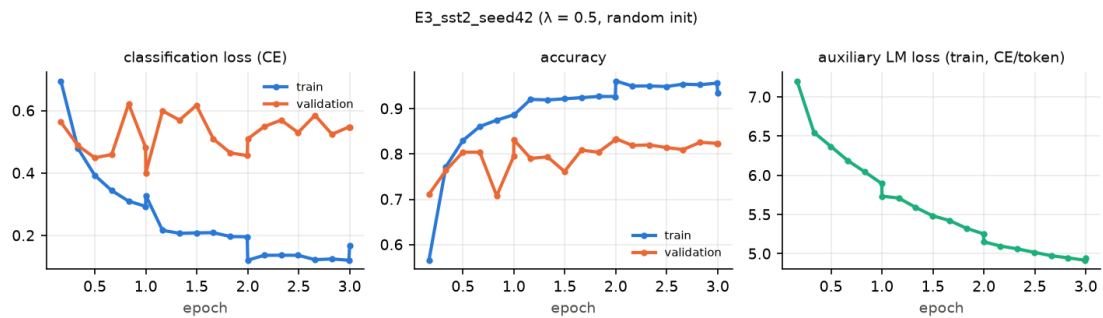


Figure 8. E3 on SST-2 (random initialisation).

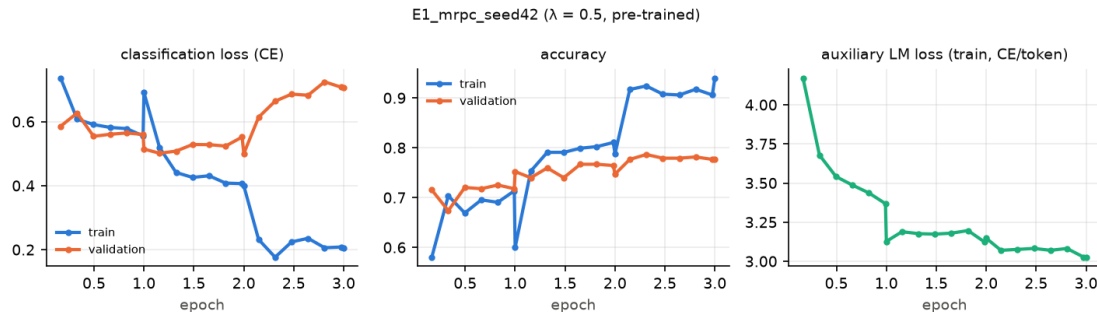


Figure 9. E1 on MRPC (seed 42).

- **SST2 (E1, seed 42):** best validation accuracy 93.00% at epoch 2.33, final 92.78%; validation loss is lowest at epoch 1.00 and ends at 0.264 (rising after its minimum, a mild sign of over-confidence). Final train–validation accuracy gap: 5.1 points. We report the final-epoch model (no checkpoint selection on the validation set).
- **MRPC (E1, seed 42):** best validation accuracy 78.68% at epoch 2.31, final 77.70%; validation loss is lowest at epoch 2.00 and ends at 0.708 (rising after its minimum, a mild sign of over-confidence). Final train–validation accuracy gap: 16.4 points. We report the final-epoch model (no checkpoint selection on the validation set).

8.4 Layer-transfer analysis (E6)

Table 8.3. Effect of the number of transferred pre-trained blocks on MRPC.

Pre-trained layers transferred (k)	Accuracy (%)	F1 positive (%)	F1 macro (%)
0	70.59	80.07	62.00
3	70.10	79.32	62.67
6	76.47	83.16	72.07
9	75.74	82.72	70.99
12	77.70	84.81	71.44
none (random embeddings too; = E3)	71.32	80.72	62.37

MRPC validation, seed 42, $\lambda = 0.5$. $k = 0$ transfers only the token/position embeddings; $k = 12$ is the full model (E1, seed 42).

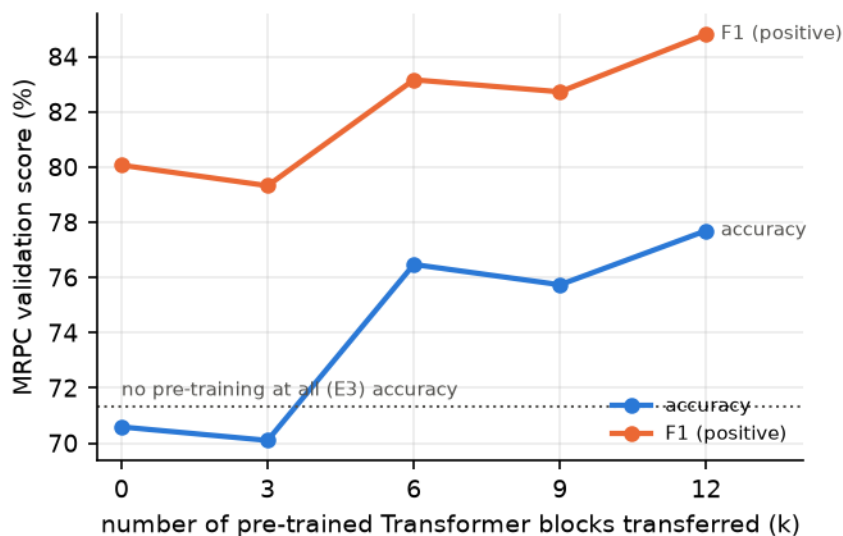


Figure 10. MRPC validation score as a function of the number of pre-trained blocks transferred.

Accuracy goes from 70.59% with $k = 0$ (only embeddings transferred) to 77.70% with $k = 12$, a gain of 7.11 points. The curve is not monotonic: with a single seed on a 408-example validation set, differences of one or two points are within noise (one example = 0.25 points), so only the overall trend should be read. For reference, training everything from scratch (E3, no pre-trained embeddings either) gives 71.32%.

8.5 Zero-shot sentiment (E5)

Table 8.4. Zero-shot SST-2 with the paper’s heuristic (§5 of the paper).

Decision rule	Accuracy (%)	F1 macro (%)	Predicted positive (%)
Paper rule: P(positive) > P(negative) after appending “very”	78.21	78.12	42.66
Diagnostic: threshold at median log-odds (not the paper’s method)	81.19	81.19	50.00
Majority-class guess	50.92	–	–

SST-2 validation, $n = 872$; true positive rate 50.92%.

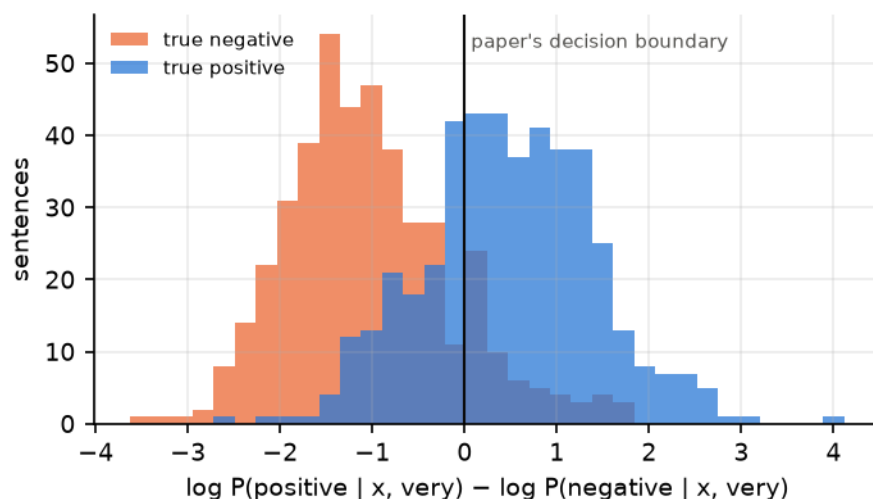


Figure 11. Zero-shot log-odds of “positive” versus “negative” after appending “very”, split by true label.

The paper’s rule gives 78.21% accuracy with no labelled data. The model predicts *positive* for 42.7% of sentences although 50.9% are positive, i.e. part of the error is a constant preference for one of the two label words rather than a lack of sentiment knowledge. Moving the threshold to the median log-odds (a diagnostic only, not the paper’s method) gives 81.19%, which shows how much sentiment information the purely generative model already carries before any fine-tuning.

8.6 Error analysis

Table 8.5. The most confident mistakes of E1 on SST-2. The last column is an automatic heuristic (negation words, contrast connectives, length), not a human judgement.

#	GLUE idx	Sentence	True	Pred	Confidence	Likely cause (automatic heuristic)
1	183	the lower your expectations , the more you ’ll enjoy it .	negative	positive	1.000	no surface cue: irony, world knowledge or debatable gold label
2	791	it ’s somewhat clumsy and too	negative	positive	1.000	contrast connective : clauses of

#	GLUE idx	Sentence	True	Pred	Confidence	Likely cause (automatic heuristic)
		lethargical ly paced – but its story about a mysteriou s creature with psychic abilities offers a solid build-up , a terrific climax , and some nice chills along the way .				opposite polarity; long sentence with mixed content
3	92	you wo n't like roger , but you will quickly recognize him .	negative	positive	1.000	negation word: polarity flip or scope misjudged; contrast connective : clauses of opposite polarity
4	118	every nanoseco nd of the the new guy reminds you that you could	negative	positive	0.999	no surface cue: irony, world knowledge or debatable gold label

#	GLUE idx	Sentence	True	Pred	Confidence	Likely cause (automatic heuristic)
5	93	be doing something else far more pleasurable . if steven soderbergh's 'solaris' is a failure it is a glorious failure . positive negative 0.999 no surface cue: irony, world knowledge or debatable gold label 6 699 ... routine , harmless diversion and little else . positive negative 0.999 rhetorica	positive	negative	0.995	contrast connective : clauses of opposite polarity

#	GLUE idx	Sentence	True	Pred	Confidence	Likely cause (automatic heuristic)
		1 question / exclamation, possibly ironic 7 271 as unseemly as its title suggests . positive negative 0.999 no surface cue: irony, world knowledge or debatable gold label 8 112 hilariously inept and ridiculous . positive negative 0.998 very little context 9 115 sam mendes has				

#	GLUE idx	Sentence	True	Pred	Confidence	Likely cause (automatic heuristic)
		become valedictorian at the school for soft landings and easy ways out . negative positive 0.998 no surface cue: irony, world knowledge or debatable gold label 10 626 this flick is about as cool and crowd- pleasing as a documenta ry can get . positive negative 0.998 no surface cue: irony, world knowledge				

#	GLUE idx	Sentence	True	Pred	Confidence	Likely cause (automatic heuristic)
		or debatable gold label 11 760 writer/director joe carnahan 's grimy crime drama is a manual of precinct cliches , but it moves fast enough to cover its clunky dialogue and lapses in logic . positive negative 0.997 contrast connective: clauses of opposite polarity 12 358 (d) oes n't bother being as cloying				

#	GLUE idx	Sentence	True	Pred	Confidence	Likely cause (automatic heuristic)
		or preachy as equivalen t evangelic al christian movies -- maybe the filmmaker s know that the likely audience will already be among the faithful . positive negative 0.997 negation word: polarity flip or scope misjudged 13 519 moretti 's compellin g anatomy of grief and the difficult process of adapting to loss . 				

#	GLUE idx	Sentence	True	Pred	Confidence	Likely cause (automatic heuristic)
		negative positive 0.996 no surface cue: irony, world knowledge or debatable gold label 14 736 american chai encourage s rueful laughter at stereotypes only an indian-american would recognize . negative positive 0.995 no surface cue: irony, world knowledge or debatable gold label 15 282				

#	GLUE idx	Sentence	True	Pred	Confidence	Likely cause (automatic heuristic)
		while there 's something intrinsically funny about sir anthony hopkins saying get in the car , bitch ,' this jerry bruckheimer production has little else to offer				

The 15 most confident errors of E1 (seed 42) out of 63 errors on 872 SST-2 validation sentences.

Among the 63 errors of E1 (seed 42), the automatic cue rates compared with the 809 correctly classified sentences are:

Cue	% of errors	% of correct predictions
negation	28.6	22.5
contrast	25.4	22.5
very short	4.8	5.3
long	3.2	5.2

Cues over-represented among errors: **negation**.

The last column of Table 8.5 gives an automatic first diagnosis of each error. The categories are the usual failure modes of sentence-level sentiment models:

1. **Contrast and concession.** Sentences of the form “X, but Y” require weighing two clauses of opposite polarity. The model has to decide which clause carries the reviewer’s overall opinion, and SST-2 annotators usually follow the clause after “but”.

2. **Negation and hedging.** “not bad”, “hardly a masterpiece” and “never boring” flip or soften the polarity of the word they modify. Errors appear when the sentiment-bearing word is strong and the negation is several tokens away.
3. **Irony, faint praise and world knowledge.** Phrases that are positive on the surface but negative in intent (or the reverse) need knowledge of film-review conventions that neither the phrase-level training data nor the BooksCorpus pre-training covers reliably.
4. **Label noise and genuinely mixed sentences.** Some “errors” are sentences whose gold label is itself debatable. SST-2 labels come from averaging several annotators’ sentiment scores and then binarising.

8.7 Attention visualisation

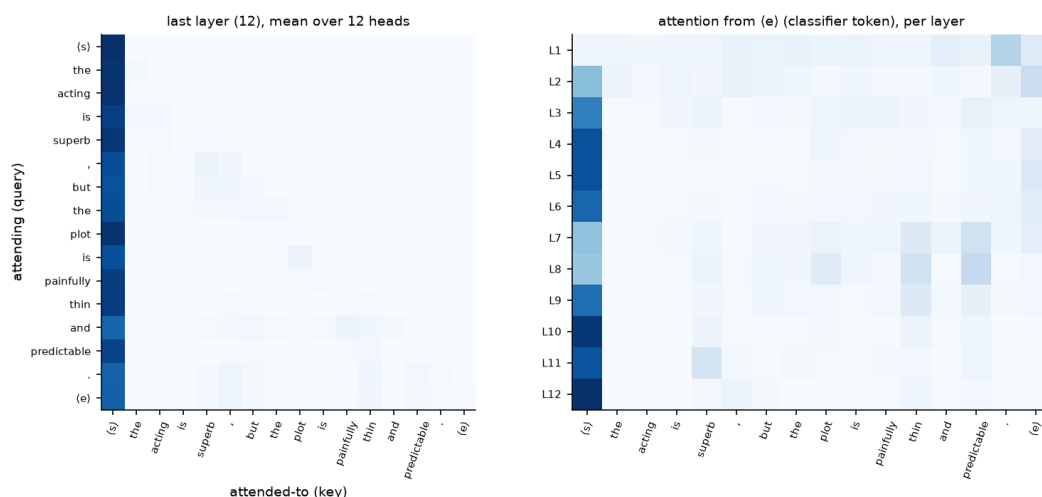


Figure 12. Attention in the fine-tuned SST-2 model (E1). Left: last layer, average over heads. Right: attention of the extract token $\langle e \rangle$ in every layer.

For the sentence “the acting is superb , but the plot is painfully thin and predictable .”, the $\langle e \rangle$ token in the last layer places its largest attention weights (averaged over heads, excluding $\langle s \rangle$ and $\langle e \rangle$) on “,” (0.05), “thin” (0.03), “predictable” (0.02), “superb” (0.02). The weight on $\langle s \rangle$ is 0.81; heavy attention to the first token is a well-known “attention sink” pattern and is not by itself evidence that the token matters. Attention weights are a diagnostic, not an explanation.

9. Comparison and discussion

Table 9.1. Our validation results next to the paper’s test results.

Setting	Paper SST-2 acc (GLUE test)	Ours SST-2 acc (dev)	Paper MRPC F1 (GLUE test)	Ours MRPC F1 (dev)
full (w/ aux LM)	91.3	92.78	82.3	85.69 $\pm$ 1.29

Setting	Paper SST-2 acc (GLUE test)	Ours SST-2 acc (dev)	Paper MRPC F1 (GLUE test)	Ours MRPC F1 (dev)
w/o aux LM	92.0	92.55	84.9	87.02 $\pm$ 1.46
w/o pre-training	84.0	82.34	79.4	80.72
LSTM w/ aux LM	90.5	not run	83.2	not run

Paper values: Radford et al. (2018), Table 4 and Table 5 (GLUE benchmark). Ours: GLUE validation split. The two are not directly comparable (different split, and see caveats).

How close are we to the paper? Several differences make an exact comparison impossible, so the table should be read as “same ballpark or not”:

- *Split.* The paper reports the hidden GLUE **test** sets. We report the public **validation** sets, which have a different size and composition (SST-2 validation has 872 sentences, so one example is 0.11 points of accuracy; MRPC validation has 408 pairs).
- *Variance.* Fine-tuning runs of this size vary by around a point between seeds, more on MRPC. The paper reports single numbers; we report means over the seeds we ran.
- *Pre-processing and tokenisation.* We use the Hugging Face port of the original ftfy + spaCy + BPE pipeline. Small differences in text cleaning are possible.
- *Implementation details not in the paper.* Gradient clipping, fp16 training, exact warm-up rounding, the treatment of special tokens in the LM loss, and the choice of the final-epoch model are our choices.
- *Per-task tuning.* The paper says it used 6.25×10^{-5} and batch 32 “for most tasks”, so a few tasks may have used other settings. We used the stated defaults everywhere.

Pre-training. In the paper (Table 5), removing pre-training causes the largest drop of any ablation (−14.8 on average). The mechanism is straightforward. A 117 M-parameter network fine-tuned from scratch on a few thousand or tens of thousands of labelled examples has far more parameters than the labels can constrain, and it cannot learn syntax, negation or world knowledge from the labels alone, whereas the pre-trained network brings them from BooksCorpus. On SST2, removing pre-training costs 10.44 points of accuracy, the largest drop of our ablations, as in the paper. On MRPC, removing pre-training costs 7.76 points of accuracy, the largest drop of our ablations, as in the paper. The layer-transfer runs (E6) refine this: going from embeddings only ($k = 0$) to all 12 blocks changes MRPC accuracy by +7.11 points, so the useful knowledge is spread over the depth of the network and not only in the embeddings, as in the paper’s Fig. 2 (left).

The auxiliary LM objective is not a free win. The paper introduces L_3 because it “improved generalization” and “accelerated convergence”, but its own ablation (Table 5) shows the auxiliary loss helps the large NLI datasets and QQP and *hurts* slightly on SST-2 (92.0 vs 91.3) and MRPC (84.9 vs 82.3). The authors conclude that “larger datasets benefit

from the auxiliary objective but smaller datasets do not”. Our E1-vs-E2 comparison tests exactly this on the two datasets we used: **SST2**: Removing the auxiliary objective ($\lambda = 0$) changes accuracy by -0.23 points (92.78 $\rightarrow$ 92.55; paper, Table 5: 91.3 $\rightarrow$ 92.0, +0.70). This is within what seed-to-seed variation alone produces (≈ 0.5 points), so on SST2 we see no measurable benefit from the auxiliary loss, consistent with the paper’s remark that smaller datasets do not benefit from it. **MRPC**: Removing the auxiliary objective ($\lambda = 0$) changes F1 by +1.34 points (85.69 $\rightarrow$ 87.02; paper, Table 5: 82.3 $\rightarrow$ 84.9, +2.60). This is within what seed-to-seed variation alone produces (≈ 1.5 points), so on MRPC we see no measurable benefit from the auxiliary loss, consistent with the paper’s remark that smaller datasets do not benefit from it. There is also a plausible mechanism for why the auxiliary loss matters less here. With only three epochs and a small learning rate, fine-tuning moves the weights little, so the knowledge that the LM loss is meant to protect is not forgotten much anyway. Its benefit as a regulariser is therefore small, while it spends part of the gradient on modelling the (short, domain-specific) task text.

Transformers against a classical baseline. TF-IDF + logistic regression is fast, CPU-only and interpretable. It sees only which n-grams occur, not how they combine, so beyond bigrams it cannot model negation scope or contrast (“not really good”, “good, but”), which is where a contextual model is expected to help. On SST2 the baseline reaches 82.11% accuracy against 92.78% for GPT-1 (+10.67), while the randomly initialised Transformer reaches 82.34% (+0.23 relative to the baseline). On MRPC the baseline reaches 75.00% accuracy against 79.08% for GPT-1 (+4.08), while the randomly initialised Transformer reaches 71.32% (-3.68 relative to the baseline). On MRPC the classical model has to rely on lexical overlap features; paraphrases with little word overlap and non-paraphrases with high overlap are its typical errors.

Zero-shot behaviour. The paper claims that the generative model learns task-relevant knowledge, such as sentiment, from books alone (§5, Fig. 2 right). With the paper’s exact heuristic our model scores 78.21% against a majority-class rate of 50.92%, i.e. above chance without ever seeing a label; with the bias-corrected threshold it reaches 81.19%, which shows that the log-odds do carry sentiment information. Fine-tuning (E1, 92.78%) remains far better. GPT-2 and GPT-3 later showed that the zero-shot gap to fine-tuned models shrinks as the pre-trained model grows.

Practical cost. Fine-tuning a 117 M-parameter model on a 4 GB laptop GPU is feasible with gradient accumulation, mixed precision, dynamic padding and a memory-aware LM loss. The effective batch size and all optimisation hyper-parameters remain those of the paper, so these engineering changes affect speed and memory, not the method.

10. Limitations and observations

- **Validation instead of test.** GLUE test labels are hidden; submitting to the GLUE server was out of scope. Our validation numbers are not a like-for-like comparison with the paper’s test numbers.

- **Seeds.** SST-2 runs use a single seed because of their length on a laptop GPU (see the compute table in Section 6.4); MRPC main runs use 42, 43, 44. Differences of about one point should not be over-interpreted.
- **No pre-training of our own.** Pre-training on BooksCorpus (100 epochs on 64×512 -token batches) is thousands of GPU-days beyond this hardware. We verified that our architecture reproduces the released model exactly and started from its weights.
- **LSTM ablation not reproduced.** The paper’s fourth ablation (a 2048-unit LSTM in the same framework) would need an LSTM pre-trained on BooksCorpus, which is not publicly available.
- **Only two of the four input transformations exercised.** Entailment and multiple-choice formatting are described and are one-line variants of the implemented ones, but were not run.
- **Hyper-parameters not re-tuned.** We used the paper’s defaults everywhere, including for E3. A randomly initialised Transformer would likely prefer a larger learning rate and more epochs, so E3 measures “the paper’s recipe without pre-training”, not the best achievable model trained from scratch.
- **Domain.** SST-2 consists of movie reviews and MRPC of news. For real customer-feedback mining, the model should be validated (and ideally fine-tuned further) on in-domain labelled data such as product or service reviews.
- **Observation: GPU thermals.** The laptop GPU ran at 80–86 °C even when idle, so long runs may be slowed by thermal throttling. This affects the timing figures, not the results.

11. Conclusion

We studied GPT-1 from the original paper and re-implemented it from scratch in PyTorch: a 12-layer, 768-wide, 12-head post-LN Transformer decoder with learned positions, GELU and a tied output layer. We verified that it reproduces the released weights’ computation to within $8.61\text{e-}05$. We fine-tuned it with the paper’s objective $L_3 = L_2 + \lambda L_1$ and input transformations on a 4 GB GPU without changing the effective batch or hyper-parameters. Checked against the paper’s four qualitative conclusions, our sentiment and paraphrase experiments give the following picture:

Paper’s conclusion	Our result
Pre-training is the most important ingredient	reproduced
Transferring more pre-trained layers helps	reproduced (overall trend)
Auxiliary LM loss gives no gain on small datasets such as SST-2/MRPC	reproduced
The pre-trained LM carries task knowledge zero-shot	reproduced (above chance)

The recipe of pre-training a large Transformer language model on unlabelled text and adapting it with minimal task-specific machinery has since become the foundation of modern NLP, and this assignment shows it working end to end on modest hardware.

References

1. A. Radford, K. Narasimhan, T. Salimans, I. Sutskever. *Improving Language Understanding by Generative Pre-Training*. OpenAI technical report, 2018.
https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf
2. A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, I. Polosukhin. Attention is all you need. *NeurIPS*, 2017.
3. P. J. Liu, M. Saleh, E. Pot, B. Goodrich, R. Sepassi, Ł. Kaiser, N. Shazeer. Generating Wikipedia by summarizing long sequences. *ICLR*, 2018.
4. T. Mikolov, I. Sutskever, K. Chen, G. Corrado, J. Dean. Distributed representations of words and phrases and their compositionality. *NeurIPS*, 2013.
5. J. Pennington, R. Socher, C. D. Manning. GloVe: Global vectors for word representation. *EMNLP*, 2014.
6. A. M. Dai, Q. V. Le. Semi-supervised sequence learning. *NeurIPS*, 2015.
7. M. E. Peters et al. Deep contextualized word representations (ELMo). *NAACL*, 2018.
8. J. Howard, S. Ruder. Universal language model fine-tuning for text classification (ULMFIT). *ACL*, 2018.
9. J. Devlin, M.-W. Chang, K. Lee, K. Toutanova. BERT: Pre-training of deep bidirectional Transformers for language understanding. *NAACL*, 2019.
10. A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever. Language models are unsupervised multitask learners (GPT-2). OpenAI, 2019.
11. T. B. Brown et al. Language models are few-shot learners (GPT-3). *NeurIPS*, 2020.
12. R. Sennrich, B. Haddow, A. Birch. Neural machine translation of rare words with subword units. *ACL*, 2016.
13. D. Hendrycks, K. Gimpel. Gaussian error linear units (GELUs). arXiv:1606.08415, 2016.
14. J. L. Ba, J. R. Kiros, G. E. Hinton. Layer normalization. arXiv:1607.06450, 2016.
15. D. P. Kingma, J. Ba. Adam: A method for stochastic optimization. *ICLR*, 2015.
16. I. Loshchilov, F. Hutter. Fixing weight decay regularization in Adam (Decoupled weight decay regularization). arXiv:1711.05101, 2017.
17. Y. Zhu et al. Aligning books and movies: Towards story-like visual explanations by watching movies and reading books (BooksCorpus). *ICCV*, 2015.
18. A. Wang, A. Singh, J. Michael, F. Hill, O. Levy, S. R. Bowman. GLUE: A multi-task benchmark and analysis platform for natural language understanding. arXiv:1804.07461, 2018.

19. R. Socher et al. Recursive deep models for semantic compositionality over a sentiment treebank (SST). *EMNLP*, 2013.
20. W. B. Dolan, C. Brockett. Automatically constructing a corpus of sentential paraphrases (MRPC). *IWP*, 2005.
21. Software: PyTorch (Paszke et al., 2019); Hugging Face Transformers (Wolf et al., 2020) and Datasets (Lhoest et al., 2021), model card openai-community/openai-gpt; scikit-learn (Pedregosa et al., 2011); spaCy; ftfy; matplotlib.

Appendix A. How to reproduce

The full instructions, with expected runtimes, are in README.md. In short:

```
# 1. environment (Windows PowerShell, Python 3.12, NVIDIA GPU)
py -3.12 -m venv .venv; .\.venv\Scripts\Activate.ps1
pip install torch==2.6.0 --index-url https://download.pytorch.org/whl/cu124
pip install -r requirements.txt; python -m spacy download en_core_web_sm
powershell -ExecutionPolicy Bypass -File scripts\download_weights.ps1 # optional: resumable weight download
```

```
# 2. smoke tests of every script (100 examples, 1 epoch)
python src\run_all.py --smoke
```

```
# 3. all experiments E1-E6, then tables, figures and this report (resumable)
python src\run_all.py
```

```
# 4. rebuild only the report from results\
python src\make_report_assets.py --attention; python src\build_report.py
```

Every table and figure in Sections 7–9 is generated from results/*.json by src/make_report_assets.py, and every number in the text by src/build_report.py. A run that has not been done appears as “not run” rather than as a value.

Appendix B. Fine-tuning curves and confusion matrices for other runs

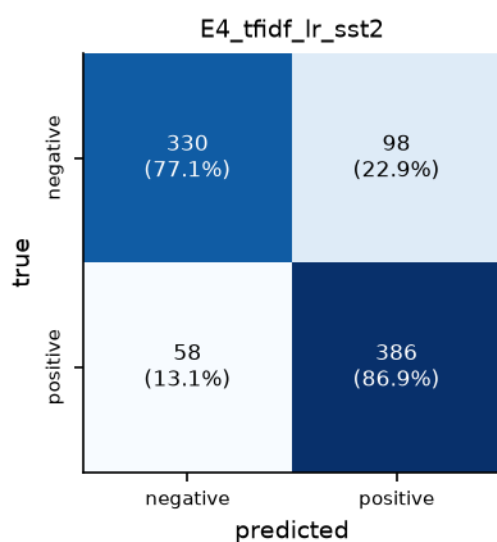


Figure B1. Confusion matrix of the TF-IDF + LR baseline on SST-2.

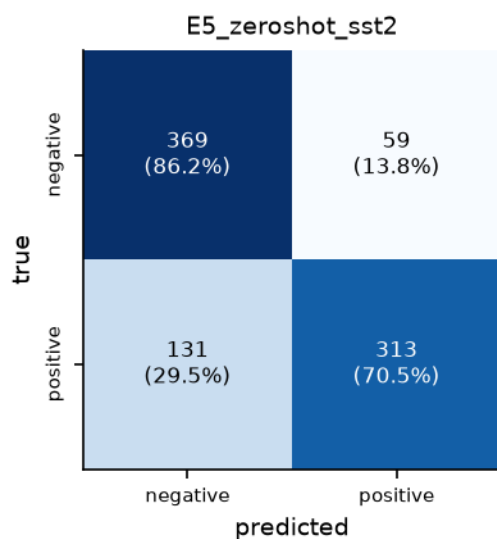


Figure B2. Confusion matrix of the zero-shot heuristic on SST-2.

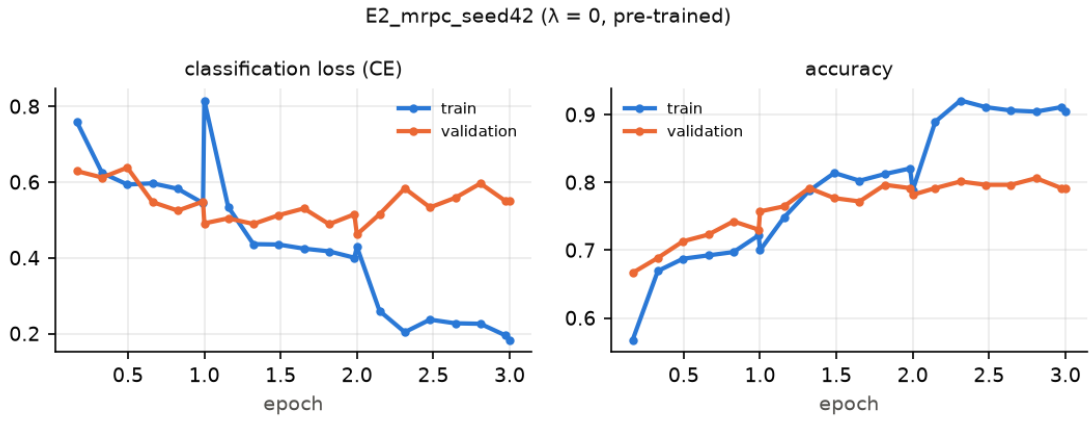


Figure B3. E2 on MRPC (seed 42).

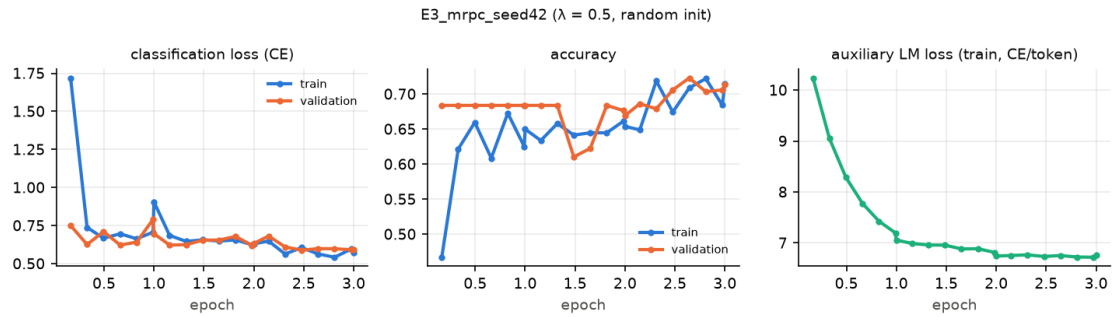


Figure B4. E3 on MRPC (random initialisation).